

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 4.3

Build a Custom Shopify MCP Server (Node.js)

Module	4 — Shopify MCP, UCP & Agentic Commerce
Topic	4.3 — re-exposing a Module 5 tool as a standalone, standards-compliant MCP server
Estimated time	30–40 minutes
Tools needed	Terminal, Node.js 20+, a code editor, Claude Code CLI
Prerequisite	A Shopify dev store with a Headless-channel Storefront private token in hand — the same one from Module 5 / Lab 3.6

What You're Building

GOAL

The exact same `search_products` logic from Lab 3.6 / Module 5, reachable by any MCP client — not just the one `query()` call that owned it before. This uses the official, LLM-agnostic **Model Context Protocol SDK** (`@modelcontextprotocol/sdk`), not the Claude Agent SDK's `createSdkMcpServer()` — that one only ever runs in-process.

By the end, Claude Code itself will be able to search your real store catalog through a tool you wrote, over a real network connection.

Prefer to vibe-code this instead? The Module 4 page has a ready-to-use Claude Code prompt covering this same build. This guide gives you the concrete code either way — to type yourself, or to compare against what Claude Code produces.

Steps 1–3: Project, Credentials, Storefront Client

1 Create the project and install dependencies

TERMINAL

```
mkdir shopify-mcp-server
cd shopify-mcp-server
npm init -y
npm pkg set type=module
npm install @modelcontextprotocol/sdk zod express dotenv
```

2 Add credentials to `.env`

```
SHOPIFY_STORE_DOMAIN=your-store.myshopify.com
SHOPIFY_STOREFRONT_PRIVATE_TOKEN=shpat_your-private-token
```

.GITIGNORE

```
node_modules
.env
```

Reusing Module 5's credential: this is the same Headless-channel Storefront private token from Lab 3.6 — read-only, and the only credential this server needs, since `search_products` never writes anything.

3 Write the Storefront client

STOREFRONTCLIENT.JS — IDENTICAL TO LAB 3.6

```
import "dotenv/config";

const domain = process.env.SHOPIFY_STORE_DOMAIN;
const token = process.env.SHOPIFY_STOREFRONT_PRIVATE_TOKEN;

export async function storefrontRequest(query, variables) {
  const res = await fetch(`https://${domain}/api/2026-07/graphql.json`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Shopify-Storefront-Private-Token": token,
    },
    body: JSON.stringify({ query, variables }),
  });
  const json = await res.json();
  if (json.errors) throw new Error(JSON.stringify(json.errors));
  return json.data;
}
```

Step 4: Build the MCP Server

4 Write `mcpServer.js`

Same tool logic as Module 5, registered through the MCP SDK's own API instead of the Claude Agent SDK's:

MCPSERVER.JS

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/streamableHttp.js";
import express from "express";
import { z } from "zod";
import { storefrontRequest } from "../storefrontClient.js";

function buildServer() {
  const server = new McpServer({ name: "shopify-tools", version: "1.0.0" });

  server.registerTool(
    "search_products",
    {
      title: "Search Products",
      description: "Search the store catalog for products matching a text query.",
      inputSchema: { query: z.string().default("") },
    },
    async ({ query }) => {
      const data = await storefrontRequest(`
        query($query: String) {
          products(first: 5, query: $query) {
            edges { node {
              title
              priceRange { minVariantPrice { amount currencyCode } }
              variants(first: 1) { edges { node { id } } }
            } }
          }
        }`, { query });

      const products = data.products.edges.map((e) => ({
        title: e.node.title,
        price: e.node.priceRange.minVariantPrice.amount,
        currency: e.node.priceRange.minVariantPrice.currencyCode,
        variantId: e.node.variants.edges[0]?.node.id ?? "",
      }));
      return { content: [{ type: "text", text: JSON.stringify({ products }) }] };
    }
  );

  return server;
}

const app = express();
app.use(express.json());
```

```
// Stateless mode: a fresh server + transport per request. Simple, and enough
// for tools with no cross-call session state (Module 5's cart tool needs a
// session id of its own if you bring it into this same server).
app.post("/mcp", async (req, res) => {
  const transport = new StreamableHTTPServerTransport({ sessionIdGenerator: undefined });
  const server = buildServer();
  await server.connect(transport);
  await transport.handleRequest(req, res, req.body);
  res.on("close", () => { transport.close(); server.close(); });
});

app.listen(3200, () => console.log("MCP server listening on :3200/mcp"));
```

Verified against a real dev store, not a mock. This exact server was run against a live Storefront API — an empty query returned the store's real unfiltered catalog (5 real products, real prices, real `gid://shopify/ProductVariant/...` IDs), and a text query for "snowboard" returned real matching products from that same store. Both requests went through a genuine MCP client (`@modelcontextprotocol/sdk` 's own `Client`), issuing a real `listTools()` handshake and a real `callTool()` round trip — not a simulated stream.

Step 5: Connect Claude Code and Run It

5 Start the server and register it with Claude Code

TERMINAL

```
node mcpServer.js
```

TERMINAL (SEPARATE WINDOW)

```
claude mcp add --transport http my-shop-tools http://localhost:3200/mcp
claude mcp list
```

`claude mcp list` should show your server as `✓ Connected`, alongside anything else you've already registered (like Topic 4.2's Storefront MCP demo).

EXPECTED OUTPUT

```
my-shop-tools: http://localhost:3200/mcp (HTTP) - ✓ Connected
```

Verified with the real Claude Code CLI, not just the SDK client. `claude mcp add --transport http` was run against this exact server, and `claude mcp list` reported it as `✓ Connected` — confirming Claude Code's own MCP client, not just a bare protocol client, can reach and handshake with the server this guide builds.

6 Ask Claude Code about your catalog

In the same Claude Code session, ask something like:

You: Do you have any snowboards?

→ Claude Code calls your `search_products` tool over the network; real titles, prices, and variant IDs come back from your dev store

What you'll see: the same kind of real catalog data Lab 3.6's terminal agent produced — except this time nothing in this conversation imported your tool code directly. Claude Code reached it purely through the MCP protocol, over HTTP.

Acceptance checklist



`claude mcp list` shows your server connected, and its tool is discoverable

- ☐ A natural-language product question in Claude Code returns real catalog data through your own tool, not Shopify's Storefront MCP

- ☐ An empty-string query still returns the unfiltered catalog, matching Lab 3.6/3.3's tested behavior

- ☐ The tool's logic is unchanged from Module 5 — only how it's reached is different

- ☐ Stopping and restarting `mcpServer.js` doesn't require re-running `claude mcp add` — the registration persists

Troubleshooting

Symptom	Likely cause	Try this
<code>claude mcp list</code> shows your server but not <code>✓ Connected</code>	<code>mcpServer.js</code> isn't running, or is listening on a different port than the URL you registered	Confirm <code>node mcpServer.js</code> is still running and printed <code>listening on :3200/mcp</code> ; re-run <code>claude mcp add</code> with the correct URL if needed
Claude Code never calls your tool, even when asked directly about products	The tool's <code>description</code> doesn't clearly say what it's for, or another connected MCP server (like Topic 4.2's Storefront MCP) is answering instead	Ask more explicitly ("use my custom search tool"), or temporarily disconnect the other server with <code>claude mcp remove</code> to confirm which one responds
A generic question like "what do you sell?" returns no products	The query arrived as <code>null</code> instead of an empty string	Confirm the zod schema is <code>z.string().default("")</code> , matching Lab 3.6/3.3's tested empty-string-vs-null behavior
The server throws on startup or every request fails	<code>SHOPIFY_STORE_DOMAIN</code> or <code>SHOPIFY_STOREFRONT_PRIVATE_TOKEN</code> is missing or wrong in <code>.env</code>	Re-check both values against the Headless channel's storefront settings from Lab 3.6
A tool call hangs and never returns	The Express route never called <code>transport.handleRequest()</code> , or the process crashed mid-request	Check the server's terminal for an uncaught error; confirm the route matches Step 4 exactly

Where This Goes Next

Keep this server running. Module 8's capstone folds it — alongside the shopping agent from Module 5, the admin-ops agent from Module 6, and the RAG support agent from Module 7 — into one multi-agent, orchestrated platform, with this MCP server as one of several standardized surfaces other agents and clients can reach.